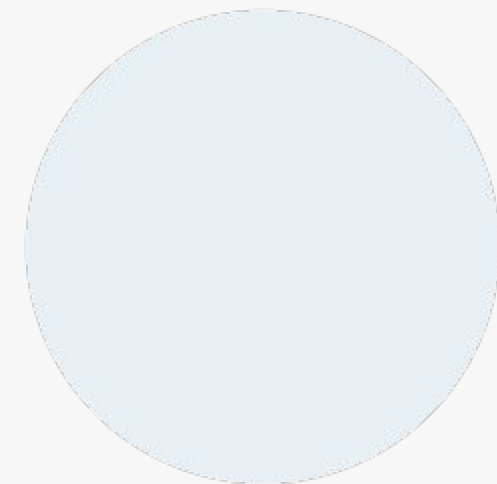


ASTON.



Локаторы. Selenium. Часть 1.



astondevs.ru

Понятие и виды локаторов



Локатор - это механизм для определения местонахождения элемента относительно HTML документа, над которым необходимо выполнять какие-то действия.

Локаторы можно расположить по приоритетам использования в следующем порядке:

- link (только для ссылок, причем при условии, что данная ссылка одна, а не серия);
- id;
- name;
- css;
- xpath.

Важным различием между CSS и XPath локаторами является то, что, используя XPath, мы можем производить перемещение как в глубину DOM иерархии, так и возвращаться назад. Что же касается CSS, то тут мы можем двигаться только в глубину. Это значит, что с XPath можем найти родительский элемент, по дочернему.

XPath



Для чего используется **XPath**

- XPath определяет синтаксис для определения частей документа XML;
- XPath использует выражения пути для навигации по элементам XML-документов;
- XPath содержит библиотеку стандартных функций;
- XPath является одним из основных элементов в XSLT;
- XPath является рекомендацией W3C.

XPath



Преимущества **XPath**

- он не уступает (или незначительно уступает) в скорости css;
- он понятен и легко читаем, по нему можно понять о каком элементе идет речь;
- он похож на язык программирования и его удобно использовать;
- можно добраться до самых заправанных элементов страницы, благодаря выстроенным цепочкам отношений;
- использовать, когда не хватает возможности простых локаторов;
- читабельный;
- универсальный - не содержит элементов, которые не влияют на нахождение элемента;
- хранить локаторы в одном месте.

Выражения XPath



Преимущества:

- Широкое распространение;
- Универсальность в рамках своего набора технологий;
- Хорошая документация и большое сообщество специалистов, с которыми можно проконсультироваться;
- Высокий уровень абстракции;
- Наличие большого набора готовых решений и описаний соответствующих лучших практик применения того или иного фреймворка для решения тех или иных задач.

Выражения XPath



XPath использует выражения пути для выбора отдельных узлов или набора узлов в документе XML. Эти выражения очень похожи на выражения, которые вы видите, когда работаете с традиционной файловой системой компьютера.

Узлы

В XPath существует семь видов узлов: элемент, атрибут, текст, пространство имён, инструкции обработки, комментарии и узлы документа. XML-документы обрабатываются в виде деревьев узлов. Верхний элемент дерева называется корневым элементом.

Атомарные значения являются узлами, не имеющие детей или родителей.

Элементы – это атомарные значения или узлы.

Отношения узлов

Рассмотрим на примере:

```
<bookstore>
  <book>
    <title>Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
</bookstore>
```

Выражения XPath



Родитель

Каждый элемент и атрибут имеет одного родителя. В следующем примере элемент «книга» (book) является родителем элементов «название» (title), «автор» (author), «год» (year) и «цена» (price)

Потомки

Узлы элементов могут иметь ноль, один или более потомков. В следующем примере элементы «название», «автор», «год» и «цена» – они все потомки элемента книга

Элементы одного уровня

Это узлы, которые имеют одного и того же родителя. В следующем примере элементы «название», «автор», «год» и «цена» все являются элементами одного уровня

Выражения XPath



Предки

Родитель узла, родитель родителя узла и т.д. В следующем примере предки элемента «название» (title) – это элементы «книга» (book) и «книжный магазин» (bookstore)

Потомки

Дети узла, дети детей узла и т.д. В следующем примере потомками элемента «книжный магазин» являются элементы «книга», «название», «автор», «год» и «цена»

Сокращённые записи XPath осей:

- attribute:: - можно заменить на "@";
- child:: - часто просто опускают;
- descendant:: - можно заменить на "///";
- parent:: - можно заменить на "..";
- self:: - можно заменить на ".".

XPath функции



- `text()` - возвращает набор текстовых нод.
- `contains()` - возвращает истину, если первая строка содержит вторую (в качестве первого значения может быть подана любая строка).
- `normalize-space()` - строковая функция убирает лишние пробелы.
- `not()` - логическое отрицание. Если рассматривать пример, то будут выбраны любые элементы, не имеющие атрибутов.
- `count()` - возвращает количество элементов.
- `last()` - возвращает номер последнего элемента в множестве.

Относительный и абсолютный путь



Простейшим маршрутом поиска является тот, который указывает на корневой узел документа (страницы). Этот маршрут представляется простой наклонной чертой / и всегда обозначает одно и то же: корневой узел документа. Каждый документ имеет только один корневой узел, являющийся общим корнем дерева узлов. Корневой узел не имеет родительских узлов. Значением корневого узла является значение элемента документа.

С наклонной черты / всегда начинается **абсолютный путь** к элементу (маршрут поиска). Получить его можно используя либо специальные расширения браузера, либо так как это делается в браузере Chrome. Вызвать окно Chrome DevTools, выделить нужный элемент, кликнув правой клавишей мыши, вызвать контекстное меню, выбрать команду Copy, а затем Copy full XPath.

Относительный и абсолютный путь



Абсолютный путь представляет собой полный и уникальный путь к элементу, начиная от корневого узла.

Абсолютный путь, по аналогии с абсолютным путем к любому файлу в файловой системе операционной системы, всегда однозначно указывает на нужный элемент. Однако у него есть один существенный недостаток: если разметка страницы изменится, то он с большой вероятностью может перестать работать. Особенно это актуально если на странице используется много различных интерактивных возможностей Javascript или анимаций элементов.

Символ / объединяет различные шаги в составной маршрут поиска. Каждый шаг в маршруте является относительным по отношению к предшествующему. Если маршрут начинается с /, то путь является абсолютным, а его первый шаг является относительным по отношению к корневому узлу.

Относительный и абсолютный путь



Относительный путь начинается с двух наклонных черт и следующим за ним одиночным тегом нужного нам элемента. Он может идентифицировать элементы в любом месте веб-страницы. И это позволяет избегать необходимости писать весь длинный абсолютный **XPath** путь, и вы можете начать его с середины структуры документа страницы (DOM). Он позволяет выбрать все элементы, по заданному тегу на странице, удовлетворяющие указанному критерию поиска.

Например, выражение **XPath** `//li` ссылается на все элементы `li` находящиеся на странице (в дереве DOM).

В начале выражения XPath символы `//` по сути позволяют выбрать всех потомков корневого узла с указанным тегом. Например, выражение XPath `//div` выбирает в документе все элементы `div`. Если мы будем использовать символ `//` в маршруте для разделения отдельных шагов, то можем опускать промежуточные шаги сокращая при этом запись маршрута. Относительный путь Xpath всегда предпочтительнее, так как он является более логичным и понятным, а также устойчивым к динамическому изменению структуры дерева DOM страницы средствами движка Javascript.

Selenium



Добавление зависимостей в проект

Для управления драйвером из клиентского кода нам нужно подключить требуемую для используемого языка программирования реализацию библиотеки WebDriver. Так как мы условились использовать сборщик Maven на текущем проекте, добавим в pom.xml, а затем в блок dependencies реквизиты библиотеки selenium-java последней стабильной версии.

```
<dependency>  
  <groupId>org.seleniumhq.selenium</groupId>  
  <artifactId>selenium-java</artifactId>  
  <version>3.141.59</version>  
</dependency>
```

На этом подключение библиотеки selenium считается завершённым. Осталось скачать сам драйвер.

Добавление зависимостей в проект



Установка конкретного драйвера

Рассматривать все тонкости работы драйверов различных браузеров мы не станем, так как эти различия незначительны. Если выбрать наиболее популярный браузер, доступный во всех ОС, то во всех примерах будет использоваться браузер Chrome.

Скачать Google Chrome на официальном сайте.
Для загрузки драйвера, совместимого с конкретной ОС, нужно перейти на официальный сайт ChromeDriver.
После распаковки архива переместим файл драйвера в папку resources.

Добавление зависимостей в проект

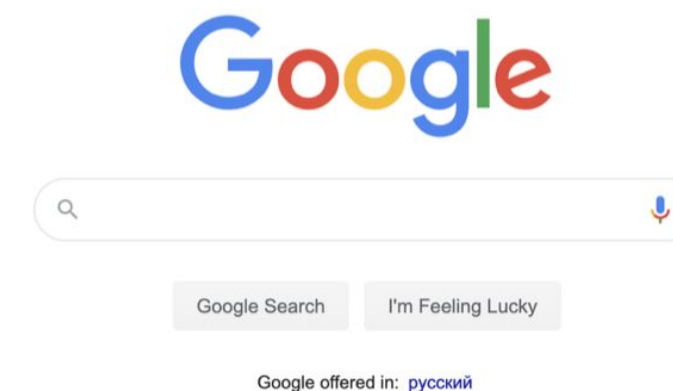
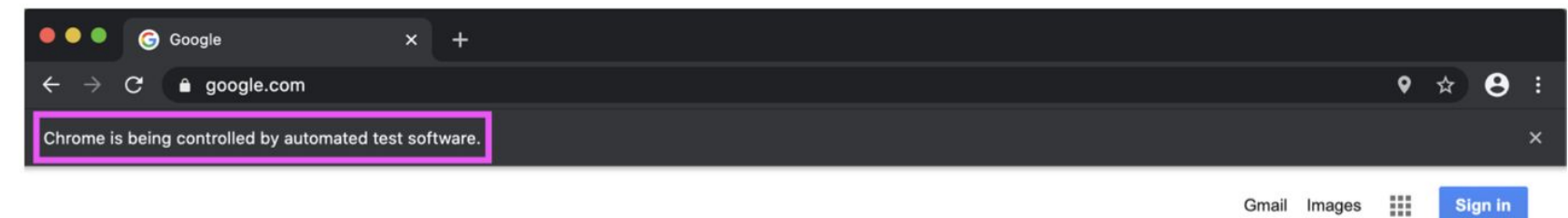


Установка конкретного драйвера

В методе main передадим в статический метод `setProperty` класса `System` идентификатор драйвера и его относительный путь. Затем инициализируем новый экземпляр `ChromeDriver` и вызовем метод `get`, передав URL в виде строки.

```
public static void main(String[] args) {  
  
    System.setProperty("webdriver.chrome.driver",  
"src/main/resources/chromedriver");  
    WebDriver driver = new ChromeDriver();  
    driver.get("https://google.com");  
}
```

Если после запуска появится новое окно Google Chrome с уведомлением, что текущая браузерная сессия находится под контролем программного обеспечения для автоматизированного тестирования — всё получилось.



Использование WebDriverManager



Способ выше отлично работает, пока используется одна версия конкретного браузера. Однако в реальном мире подобное управление драйверами сулит большими хлопотами.

В 2015 году разработчик Boni Garcia написал и опубликовал библиотеку [WebDriverManager](#), в настоящий момент активно поддерживаемую сообществом.

Разработчики отлично задокументировали этот проект, этапы подключения и настройки с примерами находятся в файле README на главной странице проекта.

Selenium Webdriver



Поиск элементов

Для поиска элементов в Selenium используются локаторы. Локатор — это строка, уникально идентифицирующая элемент страницы. В библиотеке для языка Java локаторы создаются посредством класса By — название неслучайно и соответствует семантике. WebDriver предоставляет несколько способов использования локаторов для поиска элементов.

1. By.className.
2. By.id.
3. By.name.
4. By.tagName.
5. By.linkText.
6. By.cssSelector.
7. By.xpath.

Таким образом, чтобы найти этот элемент:

```
<input name="email" class="form-control"/>
```

Используется следующая конструкция:

```
driver.findElement(By.name("email"));
```

Метод `findElement` возвращает экземпляр класса `WebElement` в случае успешного поиска, либо бросает исключение `NoSuchElementException`.

Если задача — это поиск списка элементов, следует использовать метод `findElements`.

Использование WebDriver



Основные методы драйвера

Интерфейс WebDriver содержит следующие методы для управления браузерной сессией. В приведённой ниже таблице указываются сигнатуры методов и краткое описание назначения. В рамках наших практических занятий мы рассмотрим каждый из этих методов.

Options manage();	Предоставляет интерфейс Options для управления параметрами драйвера: размер окна, ожидания, добавление или удаление cookie-файлов.
void get(String url);	Загружает веб-страницу в текущем окне.
String getCurrentUrl();	Возвращает текущий URL.
WebElement findElement(By by);	Возвращает WebElement.
List<WebElement> findElements(By by);	Возвращает список WebElements.
String getTitle();	Возвращает заголовок текущей страницы.
Set<String> getWindowHandles();	Возвращает множество дескрипторов открытых окон браузера. Используется и для итерирования.
TargetLocator switchTo();	Переключение на другой Frame или Window.
void close();	Закрывает текущее окно.
void quit();	Закрывает браузер.

Использование WebDriver



Взаимодействие с элементами

Пользователь — тот, чью работу с веб-страницей мы собираемся имитировать. С его точки зрения, наиболее часто встречаемые действия:

- клики — по кнопкам, чекбоксам, полям ввода;
- набор текста в поле ввода.

Для удовлетворения этих целей интерфейс WebElement содержит следующие методы:

```
void sendKeys(CharSequence... keysToSend);  
void click();
```

Рассмотрим гипотетический вариант использования этих методов на примере экрана авторизации:

Использование WebDriver

```
// Клик и последующий ввод
emailInputField.click();
emailInputField.sendKeys("student001@mail.ru");
// Клик и последующий ввод
passwordInputField.click();
passwordInputField.sendKeys("foobar");
// Клик
loginButton.click();
```



Вход

Вход через соцсети

vk f G GitHub Twitter @

или Email

Email

Пароль

☐ Запомнить меня

Войти

Забыли пароль? Не получили письмо для активации?

Понятие и виды ожиданий



Вследствие неустойчивого интернет-соединения или долгого ответа от сервера, нужная нам страница или элемент во время выполнения скрипта могут подгрузиться не сразу. Это может и, скорее всего, вызовет падение программы с `NoSuchElementException`.

С точки зрения программирования, простейшее решение — приостановка выполнения программы методом `Thread.sleep()`. Однако этот метод несёт в себе больше недостатков, чем достоинств. Даже если приостановить исполнение на 5 секунд, нет гарантии, что элемент не загрузится через 6. А если элемент загрузится и обнаружится мгновенно, мы вхолостую ждём эти секунды.

Они в масштабах сессии регрессионного тестирования из нескольких тысяч тестов дадут нам лишние десятки минут ожидания.

Однако, корректные способы обработки таких ситуаций давно появились.

Неявные ожидания



Неявные ожидания конфигурируют экземпляр драйвера периодически проверять наличие искомого элемента в течение обозначенного времени без выброса исключений.

```
driver.manage().timeouts().implicitlyWait(3, TimeUnit.SECONDS);
```

Здесь драйвер конфигурируется на 3 секунды ожидания. Это значит, что везде по ходу выполнения программы драйвер будет ожидать появления элемента 3 секунды и некоторое количество миллисекунд опрашивать страницу на предмет наличия искомого. Если элемент найдётся, ожидание остановится. В этом заключается ключевое отличие ожиданий драйвера от метода `sleep` класса `Thread`.

Неявные ожидания обычно настраиваются сразу после создания экземпляра `WebDriver`. Они действуют в течение всей жизни этого экземпляра, но переопределить их можно в любой момент.

Явные ожидания



Явные ожидания — Explicit Waits — это код, который ждёт наступления какого-то события, прежде чем продолжит выполнение команд скрипта. Такое ожидание срабатывает один раз в указанном месте.

```
new WebDriverWait(driver,5).until(ExpectedConditions.urlContains("/create"));
```

Пример выше в течение 5 секунд ожидает появления строки “/create” в URL текущей страницы.

Рассмотрим его более детально:

`new WebDriverWait(driver, 5)` — создание нового экземпляра класса `WebDriverWait`. В качестве параметров конструктора класс принимает текущий экземпляр драйвера и количество секунд для ожидания.

`.until(ExpectedConditions.urlContains("/create"))` — вызов метода `until` (до тех пор), принимающего в качестве параметра `ExpectedConditions` — статический класс, содержащий часто используемые условия для ожидания.

Наиболее употребляемые параметры



класса

ExpectedConditions позволяют писать самые разные условия, основанные на видимости, невидимости или кликабельности элемента, на появлении требуемого текста, изменении заголовка страницы, её URL и т. д.

- titleContains(String title);
- presenceOfElementLocated(By locator);
- presenceOfAllElementsLocatedBy(By locator);
- visibilityOfElementLocated(By locator);
- visibilityOf(WebElement element);
- textToBePresentInElement(By locator, String text);
- invisibilityOfElementLocated(By locator);
- elementToBeClickable(By locator).

Практическое задание



Необходимо написать автотесты для сайта mts.by. Суть тестов заключается в проверке блока «Онлайн пополнение без комиссии»:

1. Проверить название указанного блока;
2. Проверить наличие логотипов платёжных систем;
3. Проверить работу ссылки «Подробнее о сервисе»;
4. Заполнить поля и проверить работу кнопки «Продолжить» (проверяем только вариант «Услуги связи», номер для теста 297777777)